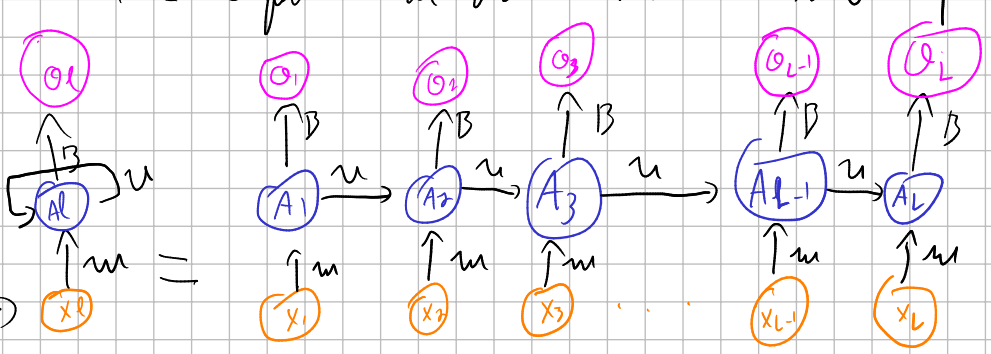
Convolution Neural Network

Recurrent Neural Network

Data is sequential → CNN will not give a good result with few resources

=> we use recurrent NN

ex: Musical sample which are sequential
CNN will do spatial analysis which isn't compatible with sequential data unlike RNN



Typer of RNN

- Serie to Serie: Like when we predict the weather we give all time series on each timestep (each day maybe)
- vector to serie: we give one image of people and sequentially predict them (one input)
- Serie to vector: One output
- Encoder-decoder: Like language translation (Eng → Fr)
  it is a combo of missing input and missing output (where there is input there is no output and vice-versa)

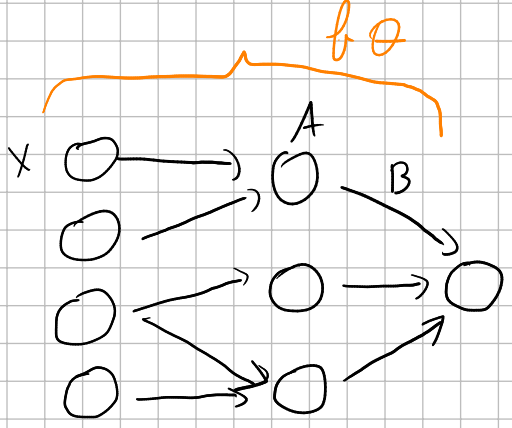
Limits: slow convergence, short memory

LSTM: Long Short-term memory

A more complex and intertwined way to construct RNN
(Look at the slides I ain't drawing that!)

Training in Neural Networks

parameters Θ to be determined



$\Theta = (B, W)$

Prediction
$X \mapsto f_\Theta(X)$
Hope:
$f_\Theta(X) = Y$

min loss function
$\rightarrow \min_{\text{new } X} \| f_\Theta(X) - Y \|^2$

$$\min_\Theta \sum_{i=1}^{n} \| f_\Theta(X_i) - Y_i \|^2 = F(\Theta)$$

Training process:

first we need to learn the data but that's out of scope

Training procedure "=" optimization problem to find Θ (up to the some preparation)

Inference procedure "=" given a data X, we use parameters Θ to compute $f_\Theta(X)$

Gradient Descent

$$\Theta^t \leftarrow \Theta^{t-1} - \gamma^t \nabla F(\theta^{t-1})$$

γ^t: step size

drawn back: computational expensive

$$\min_\Theta \sum_{i=1}^{n} \| f_\Theta(X_i) - Y_i \|^2 = F(\Theta) \rightarrow$$ gradient of $F(\theta)$ we have to apply gradient to all vectors in F

Stochastic Gradient Descent:

Repeat:

* at a time t, sample a data i uniformly random

$$\theta^t \leftarrow \theta^{t-1} - \eta^t \cdot \nabla F_i(\theta^{t-1})$$

(η^t: step size)

where $F_i(\theta^{t-1}) = \|f_{\theta^{t-1}}(x_i) - y_i\|^2$

we take only a sample randomly $\Rightarrow$ less expensive

drawback: slower convergence compared to G-D

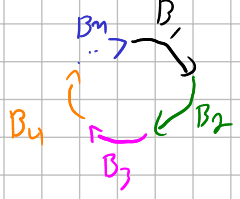
Batch Gradient Descent

Repeat:

* at time t, sample a set $B \subseteq \{1, \dots, n\}$ (size of B: to be adjusted; uniformly random)

$$\theta^t \leftarrow \theta^{t-1} - \eta^t \cdot \sum_{i \in B} \nabla F_i(\theta^{t-1})$$

But at some time we should cover all elements



round robin
Kinda thing

How can we calculate the gradient?

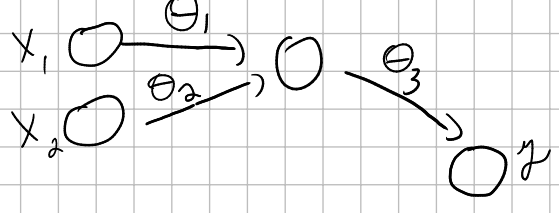
$$F(\theta) = \theta_1^2 - \theta_2 + \theta_1\theta_2$$

(θ: vector)

That is a specific example and not general for $F(\theta)$

$$\nabla F(\theta) = \left(\frac{\partial F}{\partial \theta_1}, \frac{\partial F}{\partial \theta_2}\right) = (2\theta_1 + \theta_2, \theta_1 - 1)$$

(x, y)



$$\left(g(\theta_0 + \theta_1 x_1 + \theta_2 x_2)\cdot\theta_3 - y\right)^2 = F(\theta)$$

$$\nabla F(\theta) = \left(\frac{\partial F}{\partial \theta_1}, \frac{\partial F}{\partial \theta_2}, \frac{\partial F}{\partial \theta_3}\right)$$

$$\frac{\partial F(\theta)}{\partial \theta_3} = 2\Big(\qquad\Big) \cdot g(\theta_0 + \theta_1 x_1 + \theta_2 x_2)$$

$$\frac{\partial F(\theta)}{\partial \theta_2} = 2\Big(\qquad\Big) \cdot \theta_3 \cdot g'(\theta_0 + \theta_1 x_1 + \theta_2 x_2) \cdot x_2$$

$$\frac{\partial F(\theta)}{\partial \theta_1} = 2\Big(\qquad\Big) \cdot \theta_3 \cdot g'(\theta_0 + \theta_1 x_1 + \theta_2 x_2) \cdot x_1$$

$F(\theta) = f \cdot g(\theta) \qquad \theta \in \mathbb{R}$

Chain derivation: $F'(\theta) = f'(g(\theta)) \cdot g'(\theta)$

$$\nabla F(\theta) = \left(\frac{\partial F}{\partial \theta_1}, \dots, \frac{\partial F}{\partial \theta_p}\right)$$

$$\frac{\partial F}{\partial \theta_i} = \frac{\partial F}{\partial g(\theta)} \cdot \frac{\partial g(\theta)}{\partial \theta_i}$$

$$= \frac{\partial (f(g\theta))}{\partial g(\theta)} \cdot \frac{\partial g(\theta)}{\partial \theta_i}$$

now assume the function is much more complex:

$$F = f_1(f_2(f_3(\theta)))$$

Back propagation

$$\frac{\partial F}{\partial \theta_i} = \frac{\partial f_1(\cdot)}{\partial \cdot} \cdot \frac{\partial f_2(\cdot)}{\partial \cdot} \cdot \frac{\partial f_3(\theta)}{\partial \theta} \cdot \frac{\partial \theta}{\partial \theta_i}$$

The idea of back propagation is that the derivations are connected so we can avoid recalculating stuff (save precalculation in memory).